

P1144R0

Object relocation in terms of move plus destroy

Published Proposal, 2018-10-04

Issue Tracking:

[Inline In Spec](#)

Authors:

[Arthur O'Dwyer](#)

[Mingxin Wang](#)

Audience:

LEWG, EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d1144-object-relocation.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d1144-object-relocation.html

Abstract

We define a new verb, "relocate," which is tantamount to a move and a destroy (analogous to the existing verb "swap," which is tantamount to a move, two move-assignments, and a destroy). For many C++ types, the "relocate" operation is implementable as a single `memcpy`. We provide a standard trait to detect types which are trivially relocatable, for the benefit of library writers. Finally, we provide a portable way for a user-defined type (e.g. `boost::shared_ptr`) to warrant to the implementation that it is trivially relocatable.

Table of Contents

1 Introduction and motivation

- 1.1 Optimizations enabled by trivial relocatability
- 1.2 Real-world code already does it wrong

2 Design goals

- 2.1 Standard library types such as `std::string`
- 2.2 Program-defined types that follow the Rule of Zero
- 2.3 Program-defined types with non-defaulted special members

3 Proposed language and library features

4 Proposed wording for C++20

- 4.1 Relocation operation
- 4.2 Algorithm `uninitialized_relocate`
- 4.3 Algorithm `uninitialized_relocate_n`
- 4.4 Trivially relocatable type
- 4.5 `[[trivially_relocatable]]` attribute
- 4.6 Type traits `is_relocatable` etc.
- 4.7 `Relocatable` concept

5 Further considerations and directions

- 5.1 Trivially swappable types
- 5.2 Heterogeneous relocation

6 Acknowledgements

Appendix A: Straw polls

Polls taken of SG14 on 2018-09-26

Polls requested of LEWG in San Diego

Polls requested of EWG in San Diego

Appendix B: Sample code

Defining a trivially relocatable function object

Warranting that a user-defined relocation operation is equivalent to `memcpy`

Reference implementation of `std::uninitialized_relocate`

Conditionally trivial relocation

Appendix C: Examples of non-trivially relocatable class types

Class contains pointer to self

Allocated memory contains pointer to self

Class invariant depends on `this`

Program invariant depends on `this`

Appendix D: Implementation and benchmarks

Index

Terms defined by this specification

References

Normative References

Informative References

Issues Index

§ 1. Introduction and motivation

If you are reading this paper, and you have not yet watched Arthur's session from C++Now 2018 on ["The Best Type Traits C++ Doesn't Have,"](#) you might want to [go watch](#) the first 30 minutes of that video at 2x speed with the captions turned on. It'll be worth your 15 minutes.

In the video, besides showing implementation techniques and benchmark results, we defined our terms. These terms are summarized briefly below.

C++17 knows the verbs "move," "copy," "destroy," and "swap," where "swap" is a higher-level operation composed of several lower-level operations. To this list we propose to add the verb "relocate," which is a higher-level operation composed of exactly two lower-level operations. Given an object type `T` and memory addresses `src` and `dst`, the phrase "**relocate** a `T` from `src` to `dst`" means no more and no less than "*move-construct* `dst` from `src`, and then immediately *destroy* the object at `src`."

Just as the verb "swap" produces the adjective "swappable," the verb "relocate" produces the adjective "relocatable." Any type which is both move-constructible and destructible is **relocatable**. The notion can be modified by adverbs: we say that a type is **nothrow relocatable** if its relocation operation is noexcept, and we say that a type is **trivially relocatable** if its relocation operation is trivial (which, just

like trivial move-construction and trivial copy-construction, means "the operation is tantamount to a `memcpy`").

Almost all relocatable types are trivially relocatable: `std::unique_ptr<int>`, `std::vector<int>`, `std::string`. Non-trivially relocatable types exist but are rare; see [Appendix C: Examples of non-trivially relocatable class types](#).

§ 1.1. Optimizations enabled by trivial relocatability

A reliable way of detecting "trivial relocatability" permits optimizing routines that perform the moral equivalent of `realloc`, such as

```
std::vector<R>::resize
std::vector<R>::reserve
std::vector<R>::emplace_back
std::vector<R>::push_back
```

[\[Bench\]](#) shows a 3x speedup on `std::vector<std::unique_ptr<int>>::resize`. Just as with C++11 move semantics, you can write benchmarks to show whatever speedup you like: The more complicated your types' move-constructors and destructors, the more time you save by eliminating calls to them.

A reliable way of detecting "trivial relocatability" permits optimizing routines that rely on the moral equivalent of `std::swap`, such as

```
std::swap
std::sort
std::vector<R>::insert (arguably)
```

A reliable way of detecting "trivial relocatability" permits de-duplicating the code generated by small-buffer-optimized (SBO) type-erasing wrappers such as `std::function` and `std::any`. For these types, a *move* of the wrapper object is implemented in terms of a *relocation* of the contained object. (See for example [libc++'s `std::any`](#), where the function that performs the relocation operation is confusingly named `__move`.) In general, the *relocate* operation for a contained type `C` must be uniquely codegenned for each different `C`, leading to code bloat. But a single instantiation suffices to relocate every *trivially* relocatable `C` in the program. A smaller number of instantiations means faster compile times, a smaller text section, and "hotter" code (because a relatively higher proportion of your code now fits in icache).

A reliable way of detecting "trivial relocatability" permits optimizing the move-constructor of `fixed_capacity_vector<R,N>`, which can be implemented naïvely as an element-by-element *move* (leaving the source vector's elements in their moved-from state), or can be implemented efficiently as an element-by-element *relocate* (leaving the source vector empty).

Note: `boost::container::static_vector<R,N>` currently implements the naïve element-by-element-move strategy.

Finally, some concurrent data structures might reasonably assert the trivial relocatability of their elements, just as they sometimes assert the stronger property of trivial *copyability* today.

§ 1.2. Real-world code already does it wrong

Many real-world codebases already contain templates which require trivial relocatability of their template parameters, but currently have no way to *verify* trivial relocatability. For example, [\[Folly\]](#) requires the programmer to warrant the trivial relocatability of any type stored in a `folly::fbvector`:

```
class Widget {
    std::vector<int> lst_;
};

folly::fbvector<Widget> vec; // FAILS AT COMPILE TIME for lack of warrant
```

But this merely encourages the programmer to add the warrant and continue. An incorrect warrant will be discovered only at runtime, via undefined behavior. (See [Allocated memory contains pointer to self](#) and [\[FollyIssue889\]](#).)

```
class Gadget {
    std::list<int> lst_;
};
// sigh, add the warrant on autopilot
template<> struct folly::IsRelocatable<Gadget> : std::true_type {};

folly::fbvector<Gadget> vec; // CRASHES AT RUNTIME due to fraudulent warrant
```

If this proposal is adopted, then Folly can start using

`static_assert(std::is_trivially_relocatable_v<T>)` in the implementation of `fbvector`, and the programmer can stop writing explicit warrants in the vast majority of cases.

Finally, the programmer can start writing assertions of correctness, which aids maintainability and can even find real bugs. Example:

```
class Widget {
    std::vector<int> lst_;
};
static_assert(std::is_trivially_relocatable_v<Widget>); // correctly S

class Gadget {
    std::list<int> lst_;
};
static_assert(std::is_trivially_relocatable_v<Gadget>); // correctly E
```

The improvement in user experience for real-world codebases (such as [Folly](#), [EASTL](#), BDE, Qt, etc.) is the most important benefit to be gained by this proposal.

§ 2. Design goals

Every C++ type already *is* or *is not* trivially relocatable. This proposal does not require any library vendor to make any library type trivially relocatable (but we assume that quality implementations will do so on their own).

The optimizations discussed above are purely in the domain of library writers. If you're writing a vector, and you detect that your element type `T` is trivially relocatable, then whether you do any special optimization in that case is up to you. This proposal does not require any library vendor to guarantee that any particular optimization happens (but we assume that quality implementations will do so on their own).

What C++ lacks is a standard way for library writers to *detect* the (existing) trivial relocatability of a type `T`, so that they can reliably apply their (existing) optimizations. All we really need is to add *detection*, and then all the optimizations described above will naturally emerge without any further special effort by WG21.

The following three use-cases are important for improving the performance of real programs using the standard library, and for improving the correctness of real programs using libraries such as [Folly](#)'s `fbvector`:

§ 2.1. Standard library types such as `std::string`

In order to optimize `std::vector<std::string>::resize`, we must come up with a way to achieve

```
#include <string>
static_assert(is_trivially_relocatable< std::string >::value);
```

This could be done unilaterally by the library vendor, via a non-standard attribute (`[[clang::trivially_relocatable]]`), or a member typedef with a reserved name (`using __is_triv_relocatable = void`), or simply a vendor-provided specialization of `std::is_trivially_relocatable<std::string>`.

That is, we can in principle solve §2.1 while confining our "magic" to the headers of the implementation itself. The programmer doesn't have to learn anything new, so far.

§ 2.2. Program-defined types that follow the Rule of Zero

Note: The term "program-defined types" is defined in [\[LWG2139\]](#) and [\[LWG3119\]](#).

In order to optimize the SBO `std::function` in any meaningful sense, we must come up with a way to achieve

```
#include <string>
auto lam2 = [x=std::string("hello")]{};
static_assert(is_trivially_relocatable< decltype(lam2) >::value);
```

Lambdas are not a special case in C++; they are simply class types with all their special members defaulted. Therefore, presumably we should be able to use the same solution for lambdas as for

```
#include <string>
struct A {
    std::string s;
};
static_assert(is_trivially_relocatable< A >::value);
```

Here `struct A` follows the Rule of Zero: its move-constructor and destructor are both defaulted. If they were also *trivial*, then we'd be done. In fact they are non-trivial; and yet, because the type's bases and members are all of trivially relocatable types, the type as a whole *is* trivially relocatable.

§2.2 asks specifically that we make the `static_assert` succeed without breaking the "Rule of Zero." We do not want to require the programmer to annotate `struct A` with a special attribute, or a special member typedef, or anything like that. We want it to Just Work. Even for lambda types. This is a much harder problem than §2.1; it requires standard support in the core language. But it still does not require any new *syntax*.

§ 2.3. Program-defined types with non-defaulted special members

In order to optimize `std::vector<boost::shared_ptr<T>>::resize`, we must come up with a way to achieve

```
struct B {  
    B(B&&); // non-trivial  
    ~B();  // non-trivial  
};  
static_assert(is_trivially_relocatable< B >::value);
```

via some kind of programmer-provided annotation.

Note: We cannot possibly do it without annotation, because there exist examples of types that look just like `B` and *are* trivially relocatable (for example, [libstdc++'s `std::function`](#)) and there exist types that look just like `B` and *are not* trivially relocatable (for example, [libc++'s `std::function`](#)). The compiler cannot "crack open" the definitions of `B(B&&)` and `~B()` to see if they combine to form a trivial operation. One, that's the Halting Problem. Two, the definitions of `B(B&&)` and `~B()` might not be available in this translation unit. Three, the definitions might actually be available and "crackable" in *this* translation unit, but unavailable in some other translation unit! This would lead to ODR violations and generally really bad stuff. So we cannot achieve our goal by avoiding annotation.

This use-case is the only one that requires us to design the "opt-in" syntax. In [§2.1 Standard library types such as `std::string`](#), any special syntax is hidden inside the implementation's own headers. In [§2.2 Program-defined types that follow the Rule of Zero](#), our design goal is to *avoid* special syntax. In [§2.3 Program-defined types with non-defaulted special members](#), WG21 must actually design user-facing syntax.

Therefore, I believe it would be acceptable to punt on §2.3 and come back to it later. We say, "Sure, that would be nice, but there's no syntax for it. Be glad that it works for core-language and library types. Ask again in three years." And as long as we leave the design space open, I believe we wouldn't lose anything by delaying a solution to §2.3.

This paper does propose a standard syntax for §2.3 — an attribute — which in turn provides a *simple and portable* solution to §2.1 for library vendors. However, our attribute-based syntax is severable from the rest of this paper. With extremely minor surgery, WG21 could reject our new attribute and still solve §2.1 and §2.2 for C++20.

§ 3. Proposed language and library features

This paper proposes five separate additions to the C++ Standard. These additions introduce "relocate" as a well-supported C++ notion on par with "swap," and furthermore, successfully communicate trivial relocatability in each of the three use-cases above.

- A new standard algorithm, `uninitialized_relocate(first, last, d_first)`, in the `<memory>` header.
- Additional type traits, `is_relocatable<T>` and `is_nothrow_relocatable<T>`, in the `<type_traits>` header.
- A new type trait, `is_trivially_relocatable<T>`, in the `<type_traits>` header. This is the *detection* mechanism.
- A new core-language rule by which a class type's "trivial relocatability" is inherited according to the Rule of Zero.
- A new attribute, `[[trivially_relocatable]]`, in the core language. This is the *opt-in* mechanism for program-defined types.

These five bullet points are severable to a certain degree. For example, if the `[[trivially_relocatable]]` attribute (point 5) is adopted, library vendors will certainly use it in their implementations; but if the attribute is rejected, library vendors could still indicate the trivial relocatability for certain standard library types by providing library specializations of `is_trivially_relocatable` (point 3).

Points 1 and 2 are completely severable from points 3, 4, and 5; but we believe these algorithms should be provided for symmetry with the other uninitialized-memory algorithms in the `<memory>` header (e.g. `uninitialized_move`) and the other trios of type-traits in the `<type_traits>` header (e.g. `is_destructible`, `is_nothrow_destructible`, `is_trivially_destructible`). I do not expect these templates to be frequently useful, but I believe they must be provided, so as not to unpleasantly surprise the programmer by their absence.

Points 3 and 4 together motivate point 5. In order to achieve the goal of [§2.2 Program-defined types that follow the Rule of Zero](#), we *must* define a core-language mechanism by which we can "inherit" trivial relocatability. This is especially important for the template case.

```

template<class T>
struct D {
    T t;
};

// class C comes in from outside, already marked, via whatever mechanism
constexpr bool c = is_trivially_relocatable< C >::value;
constexpr bool dc = is_trivially_relocatable< D<C> >::value;
static_assert(dc == c);

```

We strongly believe that `std::is_trivially_relocatable<T>` should be just a plain old class template, exactly like `std::is_trivially_destructible<T>` and so on. The core language *should not know or care* that the class template `is_trivially_relocatable` exists, any more than it knows that the class template `is_trivially_destructible` exists.

We expect that the library vendor will implement `std::is_trivially_relocatable`, just like `std::is_trivially_destructible`, in terms of a non-standard compiler builtin whose natural spelling is `__is_trivially_relocatable(T)`. The compiler computes the value of `__is_trivially_relocatable(T)` by inspecting the definition of `T` (and the definitions of its base classes and members, recursively, in the case that both of its special members are defaulted). This recursive process "bottoms out" at primitive types, or at any type with a user-provided move or destroy operation. For safety, classes with user-provided move or destroy operations (e.g. [Appendix C: Examples of non-trivially relocatable class types](#)) must be assumed *not* to be trivially relocatable. To achieve the goal of [§2.3 Program-defined types with non-defaulted special members](#), we must provide a way that such a class can "opt in" and warrant to the implementation that it is in fact trivially relocatable (despite being non-trivially move-constructible and/or non-trivially destructible).

In point 5 we propose that the opt-in mechanism should be an attribute. The programmer of a trivially relocatable but non-trivially destructible class `C` will mark it for the compiler using the attribute:

```

struct [[trivially_relocatable]] C {
    C(C&&); // defined elsewhere
    ~C(); // defined elsewhere
};
static_assert(is_trivially_relocatable< C >::value);

```

The attribute overrides the compiler's usual computation. An example of a "conditionally" trivially relocatable class is shown in [Conditionally trivial relocation](#).

Again, the attribute *is* severable; WG21 could adopt all the rest of this proposal and leave vendors to implement `[[clang::trivially_relocatable]]`, `[[gnu::trivially_relocatable]]`, etc., as non-standard extension mechanisms. In that case, we would strike §4.5 and one bullet point from §4.4; the rest of the proposal would remain the same.

§ 4. Proposed wording for C++20

The wording in this section is relative to [WG21 draft N4750](#), that is, the current draft of the C++17 standard.

§ 4.1. Relocation operation

Add a new section in [\[definitions\]](#):

ISSUE 1 [\[definitions\]](#) is probably the wrong place for the core-language definition of "relocation operation"

relocation operation

the homogeneous binary operation performed on a range by `std::uninitialized_relocate`, consisting of a move-construction immediately followed by a destruction of the source object

ISSUE 2 this definition of "relocation operation" is not good

§ 4.2. Algorithm `uninitialized_relocate`

Add a new section after [\[uninitialized.move\]](#):

```
template<class InputIterator, class ForwardIterator>
ForwardIterator uninitialized_relocate(InputIterator first, InputIterator last,
                                     ForwardIterator result);
```

Effects: Equivalent to:

```
for (; first != last; (void)++result, ++first) {
    ::new (static_cast<void*>(addressof(*result)))
        typename iterator_traits<ForwardIterator>::value_type(std::move(*first));
    destroy_at(addressof(*first));
}
return result;
```

§ 4.3. Algorithm `uninitialized_relocate_n`

```
template<class InputIterator, class Size, class ForwardIterator>
pair<InputIterator, ForwardIterator>
uninitialized_relocate_n(InputIterator first, Size n, ForwardIterator result);
```

Effects: Equivalent to:

```
for (; n > 0; ++result, (void) ++first, --n) {
    ::new (static_cast<void*>(addressof(*result)))
        typename iterator_traits<ForwardIterator>::value_type(std::move(*first));
    destroy_at(addressof(*first));
}
return {first, result};
```

§ 4.4. Trivially relocatable type

Add a new section in [\[basic.types\]](#):

A move-constructible, destructible object type `T` is a **trivially relocatable** type if it is:

- a trivially copyable type, or
- an array of trivially relocatable type, or
- a (possibly cv-qualified) class type declared with the `[[trivially_relocatable]]` attribute, or
- a (possibly cv-qualified) class type which:
 - has no user-provided move constructors,
 - either has at least one move constructor or has no user-provided copy constructors,
 - has no user-provided or deleted destructors,
 - either is final, or has a final destructor, or has no virtual destructors,
 - has no virtual base classes,
 - has no `mutable` or `volatile` members,
 - all of whose members are either of reference type or of trivially relocatable type, and
 - all of whose base classes are trivially relocatable.

[Note: For a trivially relocatable type, the [relocation operation](#) (such as the relocation operations performed by the library functions `std::swap` and `std::vector::resize`) is tantamount to a simple copy of the underlying bytes. —end note]

[Note: It is intended that most standard library types be trivially relocatable types. —end note]

Note: We could simplify the wording by removing the words "either is final, or has a final destructor, or". However, this would lead to the compiler's failing to identify certain (unrealistic) class types as trivially relocatable, when in fact it has enough information to infer that they *are* trivially relocatable in practice. This would leave room for a "better" implementation beneath ours. I tentatively prefer to optimize for maximum performance over spec simplicity.

Note: There is no special treatment for volatile subobjects. Using `memmove` on volatile subobjects can cause tearing of reads and writes. This paper introduces no *new* issues in this area; see [\[Subobjects\]](#). The existing issues with `volatile` are addressed narrowly by [\[P1153R0\]](#) and broadly by [\[P1152R0\]](#).

Note: There is no special treatment for possibly overlapping subobjects. Using `memmove` on possibly overlapping subobjects can overwrite unrelated objects in the vicinity of the destination. This paper introduces no *new* issues in this area; see [\[Subobjects\]](#).

ISSUE 3 The relevant move constructor, copy constructor, and/or destructor must be public and unambiguous. We imply this via the words "A move-constructible, destructible object type". However, "move-constructible" and "destructible" are library concepts, not core language concepts, so maybe it is inappropriate to use them here.

ISSUE 4 Consider the following test cases—

```
struct A {
    struct MA {
        MA(MA&);
        MA(const MA&) = default;
        MA(MA&&) = default;
    };
    mutable MA ma;
    A(const A&) = default;
};
static_assert(not std::is_trivially_relocatable_v<A>);

struct B {
    struct MB {
        MB(const volatile MB&);
        MB(const MB&) = default;
        MB(MB&&) = default;
    };
    volatile MB mb;
    B(const B&) = default;
};
static_assert(not std::is_trivially_relocatable_v<B>);

struct [[trivially_relocatable]] I {
    I(I&&);
};
struct J : I {
    J(const J&);
    J(J&&) = default;
};
static_assert(std::is_trivially_relocatable_v<J>);
```

We *must* find a rule that makes neither **A** nor **B** trivially relocatable, because the move-construction `A(std::move(a))` invokes user-provided copy constructor `MA(MA&)` and the move-construction `B(std::move(b))` invokes user-provided copy constructor `MB(const volatile MB&)`.

We would *like* to find a rule that makes **J** trivially relocatable, because the `J : I` pattern is used to implement "conditionally trivial relocatability" for all allocator-aware containers in my libc++ reference implementation. (The move-constructor and destructor of **J** are moved into a base class template **I** which is conditionally marked with `[[trivially_relocatable]]`. The copy constructor, assignment operators, etc. are not moved into the base class because they are not expected to interfere with trivial relocatability.) If the `[[trivially_relocatable]]` attribute were modified to take a boolean parameter, we might not care about this **J** example.

§ 4.5. `[[trivially_relocatable]]` attribute

Add a new section after `[dcl.attr.nouniqueattr]`:

The *attribute-token* `trivially_relocatable` specifies that a class type's relocation operation has no visible side-effects other than a copy of the underlying bytes, as if by the library function `std::memcpy`. It shall appear at most once in each *attribute-list* and no *attribute-argument-clause* shall be present. It may be applied to the declaration of a class. The first declaration of a type shall specify the `trivially_relocatable` attribute if any declaration of that type specifies the `trivially_relocatable` attribute. If a type is declared with the `trivially_relocatable` attribute in one translation unit and the same type is declared without the `trivially_relocatable` attribute in another translation unit, the program is ill-formed, no diagnostic required.

If a type `T` is declared with the `trivially_relocatable` attribute, and `T` is either not move-constructible or not destructible, the program is ill-formed.

If a class type is declared with the `trivially_relocatable` attribute, the implementation may replace relocation operations involving that type (such as those performed by the library functions `std::swap` and `std::vector::resize`) with simple copies of the underlying bytes.

If a class type is declared with the `trivially_relocatable` attribute, and the program relies on observable side-effects of relocation other than a copy of the underlying bytes, the behavior is undefined.

ISSUE 5 "If a type `T` is declared with the `trivially_relocatable` attribute, and `T` is either not move-constructible or not destructible, the program is ill-formed." We might want to replace this wording with a mere "Note" *encouraging* implementations to diagnose. See [this example](#) where a diagnostic might be unwanted.

§ 4.6. Type traits `is_relocatable` etc.

Add new entries to Table 46 in `[meta.unary.prop]`:

Template	Condition	Preconditions
<code>template<class T> struct is_relocatable;</code>	<code>is_move_constructible_v<T></code> is true and <code>is_destructible_v<T></code> is true	T shall be a complete type, cv <code>void</code> , or an array of unknown bound.
<code>template<class T> struct is_nothrow_relocatable;</code>	<code>is_relocatable_v<T></code> is true and both the indicated move-constructor and the destructor are known not to throw any exceptions.	T shall be a complete type, cv <code>void</code> , or an array of unknown bound.
<code>template<class T> struct is_trivially_relocatable;</code>	T is a <u>trivially relocatable</u> type.	T shall be a complete type, cv <code>void</code> , or an array of unknown bound.

§ 4.7. Relocatable concept

Add a new section after [\[concept.moveconstructible\]](#):

```
template<class T>
concept Relocatable = MoveConstructible<T> && Destructible<T>;
```

Note: This concept is exactly equivalent to `MoveConstructible<T>`.

§ 5. Further considerations and directions

§ 5.1. Trivially swappable types

[Mingxin Wang has proposed](#) that "swap" could be expressed in terms of "relocate". `std::swap` today is typically implemented in terms of one move-construction, two move-assignments, and one destruction; but there is nothing in the Standard that prevents a library vendor from implementing it as three relocations, which in the trivially-relocatable case (the usual case for most types) could be optimized into three calls to `memcpy`.

For reasons [described elsewhere](#), it seems reasonable to claim that move-assignment must always "do the sane thing," and therefore we might propose to define

```

template<class T>
struct is_trivially_swappable : bool_constant<
    is_trivially_relocatable_v<T> &&
    is_move_assignable_v<T>
> {};

```

thus completing the currently-incomplete trio with `is_swappable` and `is_nothrow_swappable`.

However, we do not propose "trivially swappable" at the present time. It can easily be added in a later paper.

§ 5.2. Heterogeneous relocation

Consider that `is_relocatable_v<T>` means `is_constructible_v<T, T&&>` and `is_destructible_v<T>`. We have access to a heterogeneous `is_constructible<T, Us...>`. Should we add a heterogeneous `is_relocatable_from<T, U>`?

Notice that `uninitialized_copy` and `uninitialized_move` are already heterogeneous. Here is what a heterogeneous `uninitialized_relocate` would look like.

```

template<class FwdIt, class OutIt>
void uninitialized_relocate(FwdIt first, FwdIt last, OutIt d_first) {
    using SrcT = remove_cvref_t<decltype(*first)>;
    using DstT = remove_cvref_t<decltype(*d_first)>;
    static_assert(is_relocatable_from_v<DstT, SrcT>);
    if constexpr (is_trivially_relocatable_from_v<DstT, SrcT>) {
        static_assert(sizeof (SrcT) == sizeof (DstT));
        if constexpr (is_pointer_v<FwdIt> && is_pointer_v<OutIt>) {
            // Trivial relocation + contiguous iterators = memcpy
            size_t n = last - first;
            if (n) memcpy(d_first, first, n * sizeof (SrcT));
            d_first += n;
        } else {
            while (first != last) {
                memcpy(addressof(*d_first), addressof(*first), sizeof (SrcT));
                ++d_first; ++first;
            }
        }
    } else {
        while (first != last) {
            ::new ((void*)addressof(*d_first)) DstT(move(*first));
            (*first).~SrcT();
            ++d_first; ++first;
        }
    }
    return d_first;
}

```

This implementation could be used to quickly relocate an array of `int*` into an array of `unique_ptr<int>` (but not vice versa). All we'd need is for somebody to set the value of `is_trivially_relocatable_from` appropriately for each pair of types in the program.

I think this is a very intriguing idea. The detection syntax (`is_trivially_relocatable_from`) is fairly obvious. But I don't see what the opt-in syntax would look like on a program-defined class such as `tombstone::optional`. Let's leave that problem alone for a few years and see what develops.

We could conceivably provide the detection trait today, with deliberately curtailed semantics, e.g.:

```

template<class T, class U>
struct is_trivially_relocatable_from : bool_constant<
    is_trivially_relocatable_v<T> and
    is_same_v<U, remove_cvref_t<T>>
> {};
template<class T, class U>
struct is_trivially_relocatable_from<T, U&> : is_trivially_relocatable_from<T, U> {};
template<class T, class U>
struct is_trivially_relocatable_from<T, U&&> : is_trivially_relocatable_from<T, U> {};

```

plus permission for vendors to extend the trait via partial specializations on a QoI basis:

```
template<class T>
struct is_trivially_relocatable_from<unique_ptr<T>, T*> : true_type {};
template<class T>
struct is_trivially_relocatable_from<const T*, T*> : true_type {};
struct is_trivially_relocatable_from<int, unsigned> : true_type {};
// and so on
```

However, if we do this, we may soon find that programmers are adding specializations of `is_trivially_relocatable_from` to their own programs, because they find it makes their code run faster. It will become a de-facto customization point, and we will never be able to "fix it right" for fear of breaking programmers' existing code.

Therefore, I believe that we should *not* pursue "heterogeneous" relocation operations in C++20.

Note that vendors are already free to optimize heterogeneous operations inside library algorithms, under the as-if rule. We lack a portable and generic detection trait, but vendors are presumably well aware of specific special cases that they *could* detect and optimize today—for example, a `std::copy` from an array of `int*` into an array of `const int*`, or from an array of 64-bit `long` into an array of 64-bit `long long` (see [\[TCF\]](#)). Today vendors generally choose not to perform these optimizations.

§ 6. Acknowledgements

Thanks to Elias Kosunen, Niall Douglas, and John Bandela for their feedback on early drafts of this paper.

Many thanks to Matt Godbolt for allowing me to install the prototype Clang implementation on Compiler Explorer (godbolt.org). See also [\[Announcing\]](#).

Thanks to Nicolas Lesser for his relentless feedback on drafts of this paper, and for his helpful review comments on the Clang implementation.

Thanks to Howard Hinnant for appearing with me on [\[CppChat\]](#), and to Jon Kalb and Phil Nash for hosting us.

Thanks to Pablo Halpern for [\[N4158\]](#), to which this paper bears a striking and coincidental resemblance—including the meaning assigned to the word "trivial," and the library-algorithm approach to avoiding the problems with "lame duck objects" discussed in [the final section](#) of [\[N1377\]](#).

See [discussion of N4034 at Rapperswil](#) (June 2014) and [discussion of N4158 at Urbana](#) (November 2014).

The term "relocation" is due to [\[EASTL\]](#) ([has_trivial_relocate](#)) and [\[Folly\]](#) ([IsRelocatable](#)). The same concept appears in pre-C++11 libraries under the name "movable": Qt ([Q_MOVABLE_TYPE](#)) and BSL ([IsBitwiseMoveable](#)). The same concept also appears in [\[Swift\]](#) as "bitwise movable."

Significantly different approaches to this problem have previously appeared in Rodrigo Castro Campos's [\[N2754\]](#), Denis Bider's [\[P0023R0\]](#) (introducing a core-language "relocation" operator), and Niall Douglas's [\[P1029R0\]](#) (treating relocatability as an aspect of move-construction in isolation, rather than an aspect of the class type as a whole).

§ Appendix A: Straw polls

§ Polls taken of SG14 on 2018-09-26

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
The type trait is_trivially_relocatable<T> (and its _v version) should be added to the <type_traits> header, under that exact name, as proposed in this paper.	1	20	7	1	0
We approve of a trait with the semantics of is_trivially_relocatable<T> , but not necessarily under that exact name. (For example, is_bitwise_relocatable .)	15	12	1	0	0
We approve of the general idea that user-defined classes should be able to warrant their own trivial relocatability.	25	5	2	0	0

§ Polls requested of LEWG in San Diego

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
The algorithm uninitialized_relocate(first, last, d_first) should be added to the <memory> header, as proposed in this paper.	(_)	_	_	_	_

The type trait <code>is_relocatable<T></code> (and its <code>_v</code> version) should be added to the <code><type_traits></code> header, as proposed in this paper.	(☐)	☐	☐	☐	☐
If <code>is_relocatable<T></code> is added, then we should also add <code>is_nothrow_relocatable<T></code> (and its <code>_v</code> version), as proposed in this paper.	(☐)	☐	☐	☐	☐
The type trait <code>is_trivially_relocatable<T></code> (and its <code>_v</code> version) should be added to the <code><type_traits></code> header, under that exact name, as proposed in this paper.	(☐)	☐	☐	☐	☐
We approve of a trait with the semantics of <code>is_trivially_relocatable<T></code> , but not necessarily under that exact name. (For example, <code>is_bitwise_relocatable</code> .)	(☐)	☐	☐	☐	☐
If <code>is_trivially_relocatable<T></code> is added, under that exact name, then the type trait <code>is_trivially_swappable<T></code> (and its <code>_v</code> version) should also be added to the <code><type_traits></code> header.	☐	☐	(☐)	☐	☐

§ Polls requested of EWG in San Diego

	SF	F	N	A	SA
We approve of the general idea that user-defined classes should be able to warrant their own trivial relocatability via a standard mechanism.	(☐)	☐	☐	☐	☐
We approve of the general idea that user-defined classes which follow the Rule of Zero should inherit the trivial relocatability of their bases and members.	(☐)	☐	☐	☐	☐
Nobody should be able to warrant the trivial relocatability of <code>class C</code> except for <code>class C</code> itself (i.e., we do not want to see a customization point analogous to <code>std::hash</code>).	(☐)	☐	☐	☐	☐
A class should be able to warrant its own trivial relocatability via the attribute <code>[[trivially_relocatable]]</code> , as proposed in this paper.	(☐)	☐	☐	☐	☐
A class should be able to warrant its own trivial relocatability via some attribute, but not necessarily under that exact name.	(☐)	☐	☐	☐	☐

A class should be able to warrant its own trivial relocatability as proposed in this paper, but via a contextual keyword rather than an attribute.	–	–	(–)	–	–
If a trait with the semantics of <code>is_trivially_relocatable<T></code> is added to the <code><type_traits></code> header, the programmer should be permitted to specialize it for program-defined types (i.e., we want to see that trait itself become a customization point analogous to <code>std::hash</code>).	–	–	–	–	(–)
Trivial relocatability should be assumed by default. Classes such as those in Appendix C should indicate their non-trivial relocatability via an opt-in mechanism.	–	–	–	–	(–)
To simplify Conditionally trivial relocation , if an attribute with the semantics of <code>[[trivially_relocatable]]</code> is added, it should take a boolean argument.	–	–	–	(–)	–

§ Appendix B: Sample code

§ Defining a trivially relocatable function object

The following sample illustrates [§2.2 Program-defined types that follow the Rule of Zero](#). Here `A` is a program-defined type following the Rule of Zero. Because all of its bases and members are warranted as trivially relocatable, and its move-constructor and destructor are both defaulted, the compiler concludes that `A` itself is trivially relocatable.

```
#include <string>
#include <type_traits>

// Assume that the library vendor has taken care of this part.
static_assert(std::is_trivially_relocatable_v< std::string >);

struct A {
    std::string s;
    std::string operator()(std::string t) const { return s + t; }
};

static_assert(std::is_trivially_relocatable_v< A >);
```

The following sample, involving an implementation-defined closure type, also illustrates [§2.2 Program-defined types that follow the Rule of Zero](#).

```
#include <string>
#include <type_traits>

// Assume that the library vendor has taken care of this part.
static_assert(std::is_trivially_relocatable_v< std::string >);

auto a = [s = std::string("hello")](std::string t) {
    return s + t;
};

static_assert(std::is_trivially_relocatable_v< decltype(a) >);
```

§ Warranting that a user-defined relocation operation is equivalent to `memcpy`

The following sample illustrates [§2.3 Program-defined types with non-defaulted special members](#). The rules proposed in this paper ensure that any type with non-trivial user-defined move and destructor operations will be considered non-trivially relocatable by default.

```
#include <type_traits>

struct C {
    const char *s = nullptr;
    C(const char *s) : s(s) {}
    C(C&& rhs) : s(rhs.s) { rhs.s = nullptr; }
    ~C() { delete s; }
};

static_assert(not std::is_trivially_relocatable_v< C >);

struct D : C {};

static_assert(not std::is_trivially_relocatable_v< D >);
```

The programmer may apply the `[[trivially_relocatable]]` attribute to override the compiler's default behavior and warrant (under penalty of undefined behavior) that this type is in fact trivially relocatable.


```

#include <type_traits>

struct [[trivially_relocatable]] E {
    const char *s = nullptr;
    E(const char *s) : s(s) {}
    E(E&& rhs) : s(rhs.s) { rhs.s = nullptr; }
    ~E() { delete s; }
};

static_assert(std::is_trivially_relocatable_v< E >);

struct F : E {};

static_assert(std::is_trivially_relocatable_v< F >);

```

§ Reference implementation of `std::uninitialized_relocate`

```

template<class InputIterator, class ForwardIterator>
ForwardIterator uninitialized_relocate(InputIterator first, InputIterator last,
                                     ForwardIterator result)
{
    using T = typename iterator_traits<ForwardIterator>::value_type;
    using U = std::remove_ref_t<decltype(std::move(*first))>;
    constexpr bool memcpable = (std::is_same_v<T, U> && std::is_trivially_relocatable_v<T>);
    constexpr bool both_contiguous = (std::is_pointer_v<InputIterator> && std::is_pointer_v<ForwardIterator>);

    if constexpr (memcpable && both_contiguous) {
        std::size_t nbytes = (char *)last - (char *)first;
        if (nbytes != 0) {
            std::memmove(std::addressof(*result), std::addressof(*first), nbytes);
            result += (last - first);
        }
    } else if constexpr (memcpable) {
        for (; first != last; (void)++result, ++first) {
            std::memmove(std::addressof(*result), std::addressof(*first), sizeof(T));
        }
    } else {
        for (; first != last; (void)++result, ++first) {
            ::new (static_cast<void*>(std::addressof(*result))) T(std::move(*first));
            std::destroy_at(std::addressof(*first));
        }
    }
    return result;
}

```

The code in the first branch must use `memmove`, rather than `memcpy`, to preserve the formally specified behavior in the case that the source range overlaps the destination range.

The code in the second branch, which performs one `memmove` per element, probably doesn't have much of a performance benefit, and might be omitted by library vendors.

§ Conditionally trivial relocation

We expect, but do not require, that `std::optional<T>` should be trivially relocatable if and only if `T` itself is trivially relocatable. We propose no dedicated syntax for conditional `[[trivially_relocatable]]`.

The following abbreviated implementation shows how to achieve an `optional<T>` which has the same trivial-move-constructibility as `T`, the same trivial-destructibility as `T`, and the same trivial-relocatability as `T`.

The primitives of move-construction and destruction are provided by four specializations of `optional_a`; then two specializations of `optional_b` extend `optional_a` and either do or do not apply the `[[trivially_relocatable]]` attribute; and finally the public `optional` extends the appropriate specialization of `optional_a`.

```

template<class T>
class optional :
    optional_a<T, is_trivially_relocatable_v<T>>
{
    using optional_a<T, is_trivially_relocatable_v<T>>::optional_a;
};

template<class T, bool R>
class optional_a :
    optional_b<T, is_trivially_destructible_v<T>, is_trivially_move_constructible_v<T>>
{
    using optional_b<T, is_trivially_destructible_v<T>,
        is_trivially_move_constructible_v<T>>::optional_b;
};

template<class T>
class [[trivially_relocatable]] optional_a<T, true> :
    optional_b<T, is_trivially_destructible_v<T>, is_trivially_move_constructible_v<T>>
{
    using optional_b<T, is_trivially_destructible_v<T>,
        is_trivially_move_constructible_v<T>>::optional_b;
};

template<class T, bool D, bool M>
class optional_b {
    union {
        char dummy_;
        T value_;
    };
    bool engaged_ = false;

    optional_b() = default;
    optional_b(inplace_t, Args&& args...) :
        engaged_(true), value_(std::forward<Args>(args)...) {}
    optional_b(optional_b&& rhs) {
        if (rhs.engaged_) {
            engaged_ = true;
            ::new (std::addressof(value_)) T(std::move(rhs.value_));
        }
    }
    ~optional_b() {
        if (engaged_) value_.~T();
    }
};

template<class T>
class optional_b<T, false, true> {
    union {
        char dummy_;
        T value_;
    };
};

```

```

    bool engaged_ = false;

    optional_b() = default;
    optional_b(inplace_t, Args&& args...) :
        engaged_(true), value_(std::forward<Args>(args)...) {}
    optional_b(optional_b&&) = default;
    ~optional_b() {
        if (engaged_) value_.~T();
    }
};

template<class T>
class optional_b<T, true, false> {
    union {
        char dummy_;
        T value_;
    };
    bool engaged_ = false;

    optional_b() = default;
    optional_b(inplace_t, Args&& args...) :
        engaged_(true), value_(std::forward<Args>(args)...) {}
    optional_b(optional_b&& rhs) {
        if (rhs.engaged_) {
            engaged_ = true;
            ::new (std::addressof(value_)) T(std::move(rhs.value_));
        }
    }
    ~optional_b() = default;
};

template<class T>
class optional_b<T, true, true> {
    union {
        char dummy_;
        T value_;
    };
    bool engaged_ = false;

    optional_b() = default;
    optional_b(inplace_t, Args&& args...) :
        engaged_(true), value_(std::forward<Args>(args)...) {}

    optional_b(optional_b&&) = default;
    ~optional_b() = default;
};

```

§ Appendix C: Examples of non-trivially relocatable class types

§ Class contains pointer to self

This fictional `short_string` illustrates a mechanism that can apply to any small-buffer-optimized class. `libc++'s std::function` uses this mechanism (on a 24-byte buffer) and is thus not trivially relocatable.

However, different mechanisms for small-buffer optimization exist. `libc++'s std::any` also achieves small-buffer optimization on a 24-byte buffer, without (necessarily) sacrificing trivial relocatability.

```
struct short_string {
    char *data_ = buffer_;
    size_t size_ = 0;
    char buffer_[8] = {};

    const char *data() const { return data_; }

    short_string() = default;
    short_string(const char *s) : size_(strlen(s)) {
        if (size_ < sizeof buffer_)
            strcpy(buffer_, s);
        else
            data_ = strdup(s);
    }
    short_string(short_string&& s) {
        memcpy(this, &s, sizeof(*this));
        if (s.data_ == s.buffer_)
            data_ = buffer_;
        else
            s.data_ = nullptr;
    }
    ~short_string() {
        if (data_ != buffer_)
            free(data_);
    }
};
```

§ Allocated memory contains pointer to self

`std::list` needs somewhere to store its "past-the-end" node, commonly referred to as the "sentinel node," whose `prev` pointer points to the list's last node. If the sentinel node is allocated on the heap, then `std::list` can be trivially relocatable; but if the sentinel node is placed within the `list` object itself (as happens on `libc++` and `libstdc++`), then relocating the `list` object requires fixing up the

list's last node's `next` pointer so that it points to the new sentinel node inside the destination `list` object. This fixup of an arbitrary heap object cannot be simulated by `memcpy`.

Traditional implementations of `std::set` and `std::map` also store a "past-the-end" node inside themselves and thus also fall into this category.

```
struct node {
    node *prev_ = nullptr;
    node *next_ = nullptr;
};

struct list {
    node n_;
    iterator begin() { return iterator(n_.next_); }
    iterator end() { return iterator(&n_); }
    list(list&& l) {
        if (l.n_.next_) l.n_.next_->prev_ = &n_; // fixup
        if (l.n_.prev_) l.n_.prev_->next_ = &n_; // fixup
        n_ = l.n_;
        l.n_ = node{};
    }
    // ...
};
```

§ Class invariant depends on `this`

The `offset_ptr` provided by [\[Boost.Interprocess\]](#) is an example of this category.

```
struct offset_ptr {
    uintptr_t value_;

    uintptr_t here() const { return uintptr_t(this); }
    uintptr_t distance_to(void *p) const { return uintptr_t(p) - here(); }
    void *get() const { return (void*)(here() + value_); }

    offset_ptr() : value_(distance_to(nullptr)) {}
    offset_ptr(void *p) : value_(distance_to(p)) {}
    offset_ptr(const offset_ptr& rhs) : value_(distance_to(rhs.get())) {}
    offset_ptr& operator=(const offset_ptr& rhs) {
        value_ = distance_to(rhs.get());
        return *this;
    }
    ~offset_ptr() = default;
};
```

§ Program invariant depends on `this`

In the following snippet, `struct Widget` is relocatable, but not trivially relocatable, because the relocation operation of destroying a `Widget` at point A and constructing a new `Widget` at point B has behavior that is observably different from a simple `memcpy`.

```
std::set<void*> registry;

struct registered_object {
    registered_object() { registry.insert(this); }
    registered_object(registered_object&&) = default;
    registered_object(const registered_object&) = default;
    registered_object& operator=(registered_object&&) = default;
    registered_object& operator=(const registered_object&) = default;
    ~registered_object() { registry.erase(this); }
};

struct Widget : registered_object {};
```

§ Appendix D: Implementation and benchmarks

A prototype Clang/libc++ implementation is at

- github.com/Quuxplusone/clang/tree/trivially-relocatable
- github.com/Quuxplusone/libcxx/tree/trivially-relocatable
- godbolt.org, under the name "x86-64 clang (experimental P1144)"

As observed in [CppChat] (@21:55), since the essence of this feature is to eliminate calls to user-defined functions, in principle you could make a benchmark to show any arbitrarily large amount of speedup. [Bench] shows a 3x speedup for `vector<unique_ptr<int>>::reserve`.

§ Index

§ Terms defined by this specification

relocation operation, in §4.1

trivially relocatable, in §4.4

§ References

§ Normative References

[N4750]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. [N4750: Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4750.pdf). May 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4750.pdf>

§ Informative References

[Announcing]

Arthur O'Dwyer. [Announcing "trivially relocatable"](https://quuxplusone.github.io/blog/2018/07/18/announcing-trivially-relocatable/). July 2018. URL: <https://quuxplusone.github.io/blog/2018/07/18/announcing-trivially-relocatable/>

[Bench]

Arthur O'Dwyer. [Benchmark code from "The Best Type Traits C++ Doesn't Have"](https://github.com/Quuxplusone/from-scratch/blob/095b246d/cppnow2018/benchmark-relocatable.cc). April 2018. URL: <https://github.com/Quuxplusone/from-scratch/blob/095b246d/cppnow2018/benchmark-relocatable.cc>

[Best]

Arthur O'Dwyer. [The Best Type Traits C++ Doesn't Have \(video\)](https://www.youtube.com/watch?v=MWBfmmg8-Yo). April 2018. URL: <https://www.youtube.com/watch?v=MWBfmmg8-Yo>

[Boost.Interprocess]

Ion Gaztañaga. [Mapping Address Independent Pointer: offset_ptr](https://www.boost.org/doc/libs/1_67_0/doc/html/interprocess/offset_ptr.html). 2005. URL: https://www.boost.org/doc/libs/1_67_0/doc/html/interprocess/offset_ptr.html

[Contra]

Arthur O'Dwyer. [Contra built-in library types](https://quuxplusone.github.io/blog/2018/04/15/built-in-library-types/). April 2018. URL: <https://quuxplusone.github.io/blog/2018/04/15/built-in-library-types/>

[CppChat]

Howard Hinnant; Arthur O'Dwyer. [cpp.chat episode 40: It works but it's undefined behavior](https://www.youtube.com/watch?v=8u5Qi4FgTP8). August 2018. URL: <https://www.youtube.com/watch?v=8u5Qi4FgTP8>

[EASTL]

Paul Pedriana. [N2271: EASTL — Electronic Arts Standard Template Library](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2271.html#type_traits_extensions). URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2271.html#type_traits_extensions

[Folly]

Facebook. [Folly documentation on "Object Relocation"](https://github.com/facebook/folly/blob/master/folly/docs/FBVector.md#object-relocation). URL: <https://github.com/facebook/folly/blob/master/folly/docs/FBVector.md#object-relocation>

[FollyIssue889]

Arthur O'Dwyer. [Traits.h marks std::list as trivially relocatable, but in fact it is not](https://github.com/facebook/folly/issues/889). URL: <https://github.com/facebook/folly/issues/889>

[LibcxxAny]

Eric Fiselier. [libc++ implementation of std::any \(trivially relocatable\)](https://github.com/llvm-mirror/libcxx/blob/8fdc4918/include/any#L389-L394). July 2016. URL: <https://github.com/llvm-mirror/libcxx/blob/8fdc4918/include/any#L389-L394>

[LibcxxFunction]

Howard Hinnant et al. [libc++ implementation of std::function \(non-trivially relocatable\)](https://github.com/llvm-mirror/libcxx/blob/4e7ffcaa/include/functional#L1719-L1734). URL: <https://github.com/llvm-mirror/libcxx/blob/4e7ffcaa/include/functional#L1719-L1734>

[LibstdcxxFunction]

Doug Gregor et al. [libstdc++ implementation of std::function \(trivially relocatable\)](https://github.com/gcc-mirror/gcc/blob/master/libstdc%2B%2B-v3/include/bits/std_function.h). URL: https://github.com/gcc-mirror/gcc/blob/master/libstdc%2B%2B-v3/include/bits/std_function.h

[LWG2139]

Loïc Joly. [What is a user-defined type?](https://cplusplus.github.io/LWG/issue2139). March 2012–June 2018. URL: <https://cplusplus.github.io/LWG/issue2139>

[LWG3119]

Hubert Tong. [Program-definedness of closure types](https://cplusplus.github.io/LWG/issue3119). June 2018—. URL: <https://cplusplus.github.io/LWG/issue3119>

[N1377]

Howard Hinnant; Peter Dimov; Dave Abrahams. [N1377: A Proposal to Add Move Semantics Support to the C++ Language](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2002/n1377.htm). September 2002. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2002/n1377.htm>

[N2754]

Rodrigo Castro Campos. [N2754: TriviallyDestructibleAfterMove and TriviallyReallocatable \(rev 3\)](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2008/n2754.html). September 2008. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2008/n2754.html>

[N4158]

Pablo Halpern. [N4158: Destructive Move \(rev 1\)](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4158.pdf). October 2014. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4158.pdf>

[P0023R0]

Denis Bider. [P0023R0: Relocator: Efficiently Moving Objects](http://open-std.org/JTC1/SC22/WG21/docs/papers/2016/p0023r0.pdf). April 2016. URL: <http://open-std.org/JTC1/SC22/WG21/docs/papers/2016/p0023r0.pdf>

[P1029R0]

Niall Douglas. [P1029R0: SG14 \[\[move_relocates\]\]](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1029r0.pdf). May 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1029r0.pdf>

[P1152R0]

JF Bastien. [P1152R0: Deprecating volatile](http://wg21.link/p1152r0). July 2018. URL: <http://wg21.link/p1152r0>

[P1153R0]

Arthur O'Dwyer; JF Bastien. [P1153R0: Copying volatile subobjects is not trivial](#). July 2018.
URL: <http://wg21.link/p1153r0>

[Sane]

Arthur O'Dwyer. [Thoughts on "sanely move-assignable"](#). July 2018. URL:
<https://quuxplusone.github.io/blog/2018/07/06/thoughts-on-sanely-move-assignable/>

[Subobjects]

Arthur O'Dwyer. [When is a trivially copyable object not trivially copyable?](#). July 2018. URL:
<https://quuxplusone.github.io/blog/2018/07/13/trivially-copyable-corner-cases/>

[Swift]

Michael Ilseman. [Swift ABI Stability Manifesto](#). January 2017. URL:
<https://github.com/apple/swift/blob/master/docs/ABIStabilityManifesto.md>

[TCF]

Arthur O'Dwyer. [Trivially-constructible-from](#). July 2018. URL:
<https://quuxplusone.github.io/blog/2018/07/03/trivially-constructible-from/>

[Wang]

Mingxin Wang. [Better Performance in Polymorphic Programming: Trivially Swappable](#). June 2018. URL: <https://groups.google.com/a/isocpp.org/d/msg/std-proposals/HGCHVSRwSMk/k7Ir-rmxBgAJ>

§ Issues Index

ISSUE 1 [definitions] is probably the wrong place for the core-language definition of "relocation operation" ↵

ISSUE 2 this definition of "relocation operation" is not good ↵

ISSUE 3 The relevant move constructor, copy constructor, and/or destructor must be public and unambiguous. We imply this via the words "A move-constructible, destructible object type". However, "move-constructible" and "destructible" are library concepts, not core language concepts, so maybe it is inappropriate to use them here. ↵

ISSUE 4 Consider the following test cases—

```
struct A {
    struct MA {
        MA(MA&);
        MA(const MA&) = default;
        MA(MA&&) = default;
    };
    mutable MA ma;
    A(const A&) = default;
};
static_assert(not std::is_trivially_relocatable_v<A>);

struct B {
    struct MB {
        MB(const volatile MB&);
        MB(const MB&) = default;
        MB(MB&&) = default;
    };
    volatile MB mb;
    B(const B&) = default;
};
static_assert(not std::is_trivially_relocatable_v<B>);

struct [[trivially_relocatable]] I {
    I(I&&);
};
struct J : I {
    J(const J&);
    J(J&&) = default;
};
static_assert(std::is_trivially_relocatable_v<J>);
```

We *must* find a rule that makes neither **A** nor **B** trivially relocatable, because the move-construction `A(std::move(a))` invokes user-provided copy constructor `MA(MA&)` and the move-construction `B(std::move(b))` invokes user-provided copy constructor `MB(const volatile MB&)`.

We would *like* to find a rule that makes **J** trivially relocatable, because the `J : I` pattern is used to implement "conditionally trivial relocatability" for all allocator-aware containers in my libc++ reference implementation. (The move-constructor and destructor of **J** are moved into a base class template **I** which is conditionally marked with `[[trivially_relocatable]]`. The copy constructor, assignment operators, etc. are not moved into the base class because they are not expected to interfere with trivial relocatability.) If the `[[trivially_relocatable]]` attribute were modified to take a boolean parameter, we might not care about this **J** example.

ISSUE 5 "If a type `T` is declared with the `trivially_relocatable` attribute, and `T` is either not move-constructible or not destructible, the program is ill-formed." We might want to replace this wording with a mere "Note" *encouraging* implementations to diagnose. See [this example](#) where a diagnostic might be unwanted. [↵](#)